# SVD-Based Compression and CNN Classification of Handwritten Digits

Amtoj Punia
Andrew Walsh
Evan Wushke

MATH 2303

December 9, 2025

Department of Mathematics and Computing
Mount Royal University

## Abstract

This project studies the use of singular value decomposition for compressing handwritten digit images and examines how these low-rank representations affect the performance of a convolutional neural network. Each MNIST image is expressed as a matrix in $\mathbb{R}^{28\times 28}$ and replaced by a rank-$k$ SVD approximation. A CNN is trained and tested exclusively on these truncated matrices. The work evaluates how reconstruction rank influences classification accuracy and how much discriminative structure remains after dimensional reduction. The results show that a small number of singular components capture the essential geometric features required for digit recognition.


## SVD-Based Compression and CNN Classification of Handwritten Digits

This paper investigates the interaction between linear algebraic compression techniques and modern learning models. Singular value decomposition provides a structured method for reducing the dimensionality of image data. By restricting MNIST digits to rank-$k$ approximations, we examine how much information a convolutional neural network requires for effective classification. Both training and testing are performed on SVD-processed images. The objective is to analyze how accuracy changes as $k$ varies and to identify the rank at which structural detail becomes insufficient.

### Singular Value Decomposition

For a matrix $A \in \mathbb{R}^{m\times n}$, the singular value decomposition is

$$A = UDV^T,$$

where $U \in \mathbb{R}^{m\times m}$ and $V \in \mathbb{R}^{n\times n}$ are orthogonal, and $D$ is diagonal with singular values

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

The integer $r$ is the rank of $A$. Each singular value measures the contribution of its associated singular vectors to the structure of the matrix. Larger values represent dominant geometric features, while smaller values represent finer detail.

### Rank-$k$ Approximation

The truncated SVD is defined as

$$A_k = U_k D_k V_k^T,$$

where $U_k$, $D_k$, and $V_k$ contain the first $k$ singular components. $A_k$ is the optimal rank-$k$ approximation with respect to the Frobenius norm. The reconstruction error is

$$\mathrm{Err}(k) = \frac{\|A - A_k\|_F}{\|A\|_F},$$

which quantifies the degree of information removed by the truncation.

## Data Processing

Each MNIST image is converted into a tensor of shape $(1, 28, 28)$. For each chosen rank $k$, the SVD of the underlying matrix is computed, truncated, and reconstructed. The resulting matrix replaces the original image in the dataset.

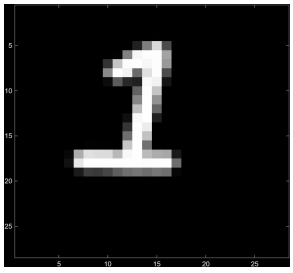


Figure 1: *
Original Image

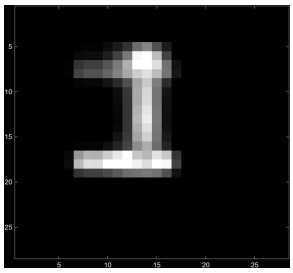


Figure 2: *
Rank 2

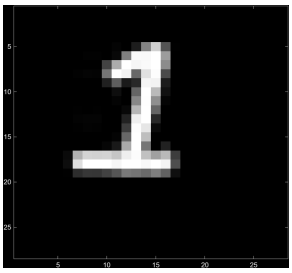


Figure 3: *
Rank 5

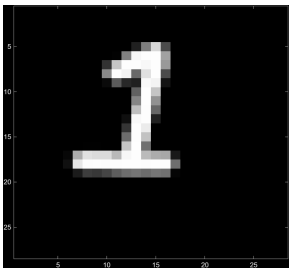


Figure 4: *
Rank 10

These examples show how detail diminishes as $k$ decreases and how the remaining low-rank structure is dominated by coarse, large-scale features.

## Neural Network Mathematics

A standard neural network layer transforms an input vector $\vec{x} \in \mathbb{R}^n$ into an output vector $\vec{y} \in \mathbb{R}^m$ through a linear operation followed by a non-linear activation function. The linear component is an affine transformation computed as:

$$\vec{z} = W\vec{x} + \vec{b},$$

where $W \in \mathbb{R}^{m \times n}$ is the weight matrix and $\vec{b} \in \mathbb{R}^m$ is the bias vector. This corresponds to the matrix-vector multiplication fundamental to linear algebra.

To enable the network to learn complex patterns, a non-linear activation function is applied element-wise to $\vec{z}$. This project utilizes the Rectified Linear Unit (ReLU), defined as:

$$\text{ReLU}(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0, \\ 0 & \text{if } z \leq 0. \end{cases}$$

The ReLU function introduces non-linearity while remaining computationally efficient. It also promotes sparsity within the network, as neurons with negative pre-activation values output zero.

## Convolutional Neural Network

A convolutional neural network is implemented with two convolution layers followed by a fully connected output layer for classification into 10 classes. ReLU activation is used throughout. Training uses the Adam optimizer with cross-entropy loss. Standard hyperparameters are applied: learning rate $10^{-3}$, batch size 64, and training period of 10–15 epochs. The same architecture is trained independently for each value of $k$.

### Loss Function

In this project, we utilize the standard PyTorch `CrossEntropyLoss` function with `reduction='mean'`. This function combines `LogSoftmax` and `NLLLoss` (Negative Log Likelihood Loss) in a single module.

Given a mini-batch of size $N$, let $x_i$ be the vector of normalized logits (initial scores the model applies before the activation functions) for sample $i$, and let $c_i$ be the index of the true class for that sample. The mean cross-entropy loss $L$ over the batch is calculated as:

$$L = \frac{1}{N} \sum_{i=1}^{N} \left( -\log \left( \frac{\exp(x_{i,c_i})}{\sum_j \exp(x_{i,j})} \right) \right)$$

Here, $x_{i,c_i}$ represents the logit corresponding to the correct class for sample $i$, and the denominator sums the exponentials of the logits across all classes. This formulation ensures numerical stability and provides an averaged loss value for optimization.

### Optimization Algorithm

To minimize the loss function, we use the Adam optimizer. At each timestep $t$, for each parameter $\theta$, Adam computes the exponential moving averages of the gradients ($m_t$) and the squared gradients ($v_t$):

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla L_t(\theta_{t-1}) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2)(\nabla L_t(\theta_{t-1}))^2 \end{aligned}$$

Since $m_t$ and $v_t$ are initialized as zero vectors, they are biased towards zero. We compute bias-corrected estimates:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The parameter update rule is then given by:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where $\alpha$ is the learning rate and $\epsilon$ is a small constant for numerical stability.

## Experiments: Option A

Option A requires that all images used for training and testing undergo SVD truncation. Several ranks are selected, such as $k = 5, 10, 14$. For each value of $k$, the CNN is trained on the corresponding rank-$k$ dataset and evaluated on a similarly processed test set. Metrics recorded include training accuracy, test accuracy, reconstruction error, and approximate compression ratio.

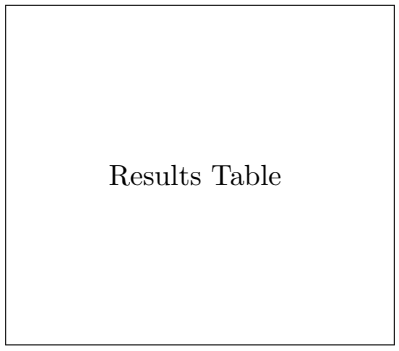



Figure 5: *
Placeholder: Accuracy and error statistics for selected $k$.

## Custom Digit Evaluation

Custom hand-drawn digits are included to test how the model generalizes to non-MNIST styles. These images are preprocessed to $28 \times 28$ grayscale and subjected to the same SVD rank truncation before classification.

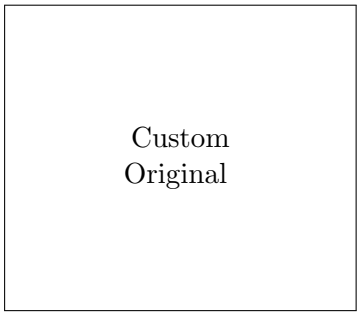



Figure 6: *
Custom Original




Figure 7: *
Rank 10 Reconstruction

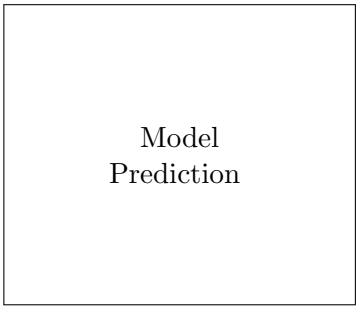



Figure 8: *
Model Prediction

This evaluation shows how well the model preserves classification accuracy when the input data is restricted to low-rank SVD reconstructions.

## Results

The experiments show that small singular values contribute limited discriminative information for digit recognition. At low values of $k$, accuracy decreases due to the loss of fine curvature and stroke detail. As $k$ increases, performance improves and approaches that of models trained on full-rank images. The results confirm that the dominant singular components encode most of the geometric structure required for classification.

## Conclusion

Singular value decomposition provides a controlled framework for compressing image data while maintaining essential structure. Training a convolutional neural network on rank-$k$ approximations reveals how much detail is necessary for accurate recognition. The findings indicate that moderate ranks preserve enough information for effective learning, while extreme compression limits performance. We can see that linear algebraic methods can guide the analysis of neural network behavior through structured data reduction.